



*Vicerrectoría Académica
Dirección de Servicios Académicos
Subdirección de Servicios a Escuelas*

**[Vecinos Conectados]
(DAS) Documento Arquitectura de Software
Versión 2.2**

Identificación de Documento

Identificación	INF-VC-01
Proyecto	Vecinos Conectados
Versión	2.2

Documento mantenido por	Dairys Bernal, Rocío Bustos y Amaru Burdiles
Fecha de última revisión	17 de mayo del 2026
Fecha de próxima revisión	18 de mayo del 2026

Documento aprobado por	Dairys Bernal, Amaru Burdiles, Rocío Bustos
Fecha de última aprobación	17 de mayo del 2026

Historia de Revisiones

Fecha	Versión	Descripción	Autor
30 de marzo del 2026	1.0	Se desarrollaron los puntos 1.1 al 1.4	Dairys Bernal
01 de abril del 2026	1.1	Se revisaron los puntos anteriores y se ejecutaron los puntos 1.5 al 2.2	Rocío Bustos
03 de abril del 2026	1.2	Se abordaron los puntos 2.3 al 2.5	Amaru Burdiles
05 de abril del 2026	1.3	Se realizo una revisión completa de todos los puntos a abordar del documento	Dairys Bernal
06 de abril del 2026	1.4	Se modifiko formato del informe.	Rocío Bustos
06 de abril del 2026	1.5	Revisión completa	Amaru Burdiles Dairys Bernal Rocío bustos
20 de abril del 2026	1.6	Se agrego una breve descripción a los requerimientos	Dairys Bernal Amaru Burdiles Rocio Bustos
4 de mayo del 2026	1.7	Se desarrollaron los puntos 4.11 y 4.1.2	Dairys Bernal Amaru Burdiles Rocio Bustos
5 de mayo del 2026	1.8	Se realizo caso de uso general y caso de uso especifico, lista de caso de uso	Dairys Bernal Amaru Burdiles Rocio Bustos
11 de mayo del 2026	1.9	Se desarrollo diagrama de clase y diagrama de paquetes	Dairys Bernal Amaru Burdiles Rocio Bustos

12 de mayo del 2026	2.0	Se desarrollo diagrama de actividad, diagrama de componentes y se corrigieron faltas ortográficas	Dairys Bernal Amaru Burdiles Rocio Bustos
16 de mayo del 2026	2.1	Se desarrollo diagrama de despliegue y se corrigieron saltos de línea del documento	Amaru Burdiles Dairys Bernal Rocio Bustos
17 de mayo del 2026	2.2	Se modificaron las descripciones del diagrama de clase y diagrama de componentes	Dairys Bernal Amaru Burdiles Rocio Bustos

Tabla de Contenidos

1. INTRODUCCIÓN	4
1.1. CONTEXTO DEL PROBLEMA	4
1.2. PROPÓSITO	4
1.3. ÁMBITO	4
1.4. DEFINICIONES, ACRÓNIMOS Y ABREVIACIONES	4
1.5. RESUMEN EJECUTIVO	4
1.6. ARQUITECTURA DEL SISTEMA	4
2. VISIÓN DEL SISTEMA	4
2.1. DESCRIPCIÓN GENERAL DEL SISTEMA	4
2.2. OBJETIVOS DEL SISTEMA	4
2.3. REQUERIMIENTOS FUNCIONALES	4
2.4. REQUERIMIENTOS NO FUNCIONALES	5
2.5. SUPUESTOS Y DEPENDENCIAS	5
3. ESTILOS Y PATRONES ARQUITECTÓNICOS	5
3.2. JUSTIFICACIÓN DEL ESTILO SEGÚN EL CONTEXTO DEL SISTEMA	5
4. MODELO 4 +1 Y VISTAS ARQUITECTÓNICAS	5
4.1. VISTA DE ESCENARIO	5
4.1.1. <i>Propósito</i>	5
4.1.2. <i>Actores</i>	5
4.1.3. <i>Diagrama general de casos de uso</i>	5
4.1.4. <i>Diagrama de casos de uso específicos</i>	5
4.1.6. <i>Especificación de casos de uso</i>	5
4.2. VISTA LÓGICA	6
4.2.1. <i>Propósito</i>	6
4.2.2. <i>Diagrama de clases</i>	6
4.2.3. <i>Descripción diagrama de clases</i>	6
4.3. VISTA DE IMPLEMENTACIÓN/DESARROLLO	7
4.3.1. <i>Propósito</i>	7

4.3.2. Diagrama de componente.....	7
4.3.3. Descripción diagrama de componente	7
4.3.4. Diagrama de paquete	7
4.3.5. Descripción diagrama de paquete	7
4.4. VISTA DE PROCESOS	7
4.4.1. PROPÓSITO	7
4.4.2. DIAGRAMA DE ACTIVIDAD	7
4.4.3. DESCRIPCIÓN DIAGRAMA DE ACTIVIDAD.....	7
4.5. VISTA FÍSICA	7
4.5.1. Propósito	7
4.5.2. Diagrama de despliegue	7
4.5.3. Descripción diagrama de despliegue	7
5. REQUISITOS DE CALIDAD	7
5.1. PROPÓSITO	7
5.3. Reglas y criterios de evaluación de calidad	7
6. PRINCIPIOS DE DISEÑO APLICADOS	8
5.2. Propósito	8
5.3. PRINCIPIOS DE DISEÑO (POR EJEMPLO: ABSTRACCIÓN, ACOPLAMIENTO, COHESIÓN, ENCAPSULAMIENTO, MODULARIDAD)	8
7. PROTOTIPO.....	8
7.1. PROPÓSITO	8
7.2. MOCKUPS (IMÁGENES CON UNA BREVE DESCRIPCIÓN)	8
7.3. JUSTIFICAR HERRAMIENTAS DE PROTOTIPADO	8
8. EVALUACIÓN DE CALIDAD HEURÍSTICA DE NIELSEN	8
8.1. PROPÓSITO	8
8.2. LISTA DE VERIFICACIÓN	8
8.3. ANÁLISIS Y MÉTRICAS DE RESULTADOS	8 9.
CONTROL DE VERSIONES	8
9.1. PROPÓSITO	8
9.2. CONTROL DE VERSIÓN UTILIZADO (JUSTIFICAR EL TIPO DE CONTROL DE VERSIÓN UTILIZAD (FECHA, SEMÁNTICA O SECUENCIAL)	8
9.3. JUSTIFICAR HERRAMIENTAS DE VERSIONAMIENTO	8
7. CONCLUSIONES.....	8
8. BIBLIOGRAFÍA	8
9. ANEXOS	8
9.1. CARTA GANTT.....	8

1. INTRODUCCIÓN

1.1. Contexto del Problema:

El edificio "El Mirador", una estructura de 20 pisos y 160 departamentos situada en una zona céntrica, enfrenta una crisis en la gestión de sus gastos comunes. Actualmente, la administración depende de procesos manuales ineficientes que han derivado en una alta tasa de morosidad, falta de liquidez y opacidad en el uso de los fondos. La desproporción entre el personal disponible y la cantidad de residentes agrava la capacidad de respuesta ante imprevistos, deteriorando la convivencia y la seguridad de la comunidad.

1.2. Propósito:

Este documento tiene como propósito definir el marco arquitectónico para el desarrollo de una solución tecnológica integral. Se busca establecer las directrices técnicas que aseguren la automatización de cobros, la transparencia informativa y la eficiencia operativa, mitigando los riesgos financieros y legales asociados a la gestión actual.

1.3. Ámbito:

El sistema abarcará la gestión automatizada de residentes (propietarios y arrendatarios), el procesamiento de pagos de gastos comunes y extraordinarios, la emisión de notificaciones en tiempo real y la administración de solicitudes o reclamos. Asimismo, integrará un módulo de control y registro de visitas para fortalecer la seguridad perimetral, permitiendo la autorización anticipada y el monitoreo de ingresos. La solución será accesible tanto para la administración como para los residentes a través de una interfaz intuitiva, asegurando una trazabilidad completa de todas las interacciones dentro de la comunidad.

1.4. Definiciones, acrónimos y abreviaciones

ACRONIMO	DESCRIPCION
<i>DAS</i>	Documento de arquitectura de software: Documento técnico que describe el diseño y la estructura del sistema.
<i>GGCC</i>	Gastos comunes: cobros mensuales por servicios y mantención.
<i>CU</i>	Caso de Uso: Descripción de una secuencia de interacciones entre el sistema y sus actores.
<i>UI/UX</i>	Interfaz de usuario/ Experiencia de usuario: Relacionado con el diseño intuitivo requerido para residentes y administración.
<i>SQL</i>	Structure Query Language: Lenguaje utilizado para la gestión y consulta de las bases de datos relacionales del edificio.
<i>PDV</i>	Punto de Venta: Referencia a los centros o mecanismos de recaudación de deudas.
<i>4+1</i>	Modelo de vistas: metodología utilizada para describir la arquitectura mediante cinco vistas (escenario, lógica, desarrollo, procesos y física).

1.5. Resumen ejecutivo

El proyecto propone la implementación de una plataforma digital centralizada que sustituya los procesos manuales por flujos automatizados. Mediante la integración de módulos de cobranza, comunicación y seguridad, se espera reducir la morosidad, optimizar el flujo de caja y restaurar la confianza de los residentes a través de un acceso transparente a la información financiera en tiempo real.

1.6. Arquitectura del sistema

La arquitectura se fundamenta en el **Modelo de Vistas 4+1**, permitiendo una comprensión integral del software desde diversas perspectivas:

- **Vista de Escenario:** Define la funcionalidad mediante casos de uso.
- **Vista Lógica:** Describe la estructura de clases y el modelo de datos.
- **Vista de Proceso:** Modela el comportamiento y flujo del sistema.
- **Vista de Desarrollo:** Organiza los componentes y paquetes de software.
- **Vista Física:** Detalla la topología del hardware y despliegue en la nube.

2. VISIÓN DEL SISTEMA

2.1. Descripción general del sistema

El sistema se concibe como una solución omnicanal que permite a los residentes visualizar sus deudas, realizar pagos electrónicos y gestionar requerimientos desde una interfaz única (con funciones como "Pagar", "Instalaciones" y "Mensajes"). Para la administración, actúa como un panel de control para la generación de informes, supervisión de personal y seguimiento de morosidades.

2.2. Objetivos del sistema

- **Automatizar** el ciclo de facturación y cobranza de gastos comunes.
- **Garantizar la transparencia** financiera mediante reportes descargables para los copropietarios.
- **Centralizar la comunicación** y la gestión de reclamos para mejorar la trazabilidad de las respuestas.
- **Reducir la morosidad** mediante recordatorios automáticos y sistemas de pago integrados.
- **Elevar los estándares de seguridad** del recinto mediante la implementación de un módulo de gestión de visitas, permitiendo un control riguroso y preventivo de los accesos al edificio.

2.3. Requerimientos funcionales

- **RF-01 Registro y gestión de residentes**

El sistema debe permitir al administrador registrar, modificar y dar de baja los datos de todos los residentes, diferenciando entre propietarios y arrendatarios. Cada residente debe quedar estrictamente vinculado a uno de los 160 departamentos del edificio.

- **RF-02 Registro y gestión de personal**

El sistema debe permitir al administrador registrar, modificar y dar de baja los datos del personal contratado por el edificio, asignándoles un rol y credenciales de acceso a la plataforma.

- **RF-03 Gestión de permisos y roles de usuario**

El sistema debe permitir al administrador gestionar el nivel de acceso de cada cuenta, asignando o modificando los roles predefinidos (Administrador, Propietario, Arrendatario, Conserje, Mantenimiento) para controlar qué módulos de la plataforma puede ver o editar cada persona.

- **RF-04 consultar los datos de residente**

El sistema debe contar con un buscador general que permita al administrador y a los conserjes consultar rápidamente la información de contacto y la unidad asociada de cualquier residente activo en el edificio.

- **RF-05 Registro y gestionar de departamentos**

El sistema debe permitir al administrador registrar e identificar cada una de las 160 unidades (departamentos) que componen el edificio, estableciendo sus características físicas y parámetros de cobro.

- **RF-06 Registro y gestión de contratos de arriendo**

El sistema debe permitir registrar formalmente el vínculo de arriendo entre un inquilino y un departamento, adjuntando el documento de respaldo y definiendo quién es el responsable legal de pagar los gastos comunes.

- **RF-07 Registro fotográfico de condiciones del dpto.**

El sistema debe permitir subir y almacenar evidencia fotográfica del estado físico del departamento (inventario visual) al momento de iniciar un nuevo contrato de arriendo o tras realizar una reparación importante.

- **RF-08 Registro y configuración de fecha de cobro**

El sistema debe permitir al administrador definir y modificar el ciclo de facturación mensual, estableciendo la fecha exacta de emisión de las cuotas y la fecha límite de pago para los residentes.

- **RF-09 Registro y validación de pagos**

El sistema debe permitir registrar los pagos de gastos comunes realizados por los residentes, ya sea validando comprobantes de transferencia manuales o registrando transacciones automáticas, para actualizar el saldo de la deuda en tiempo real.

- **RF-10 Consultar fecha de vencimiento de próxima cuota**

El sistema debe mostrar al residente de forma clara y accesible la fecha límite para el pago de su próxima cuota de gastos comunes, calculando los días restantes para prevenir atrasos.

- **RF-11 Cálculo automático de GGCC mensuales**

El sistema debe automatizar el cálculo de la cuota mensual de gastos comunes para cada departamento, sumando los egresos totales del periodo (luz, agua, gas, remuneraciones, mantenciones) y prorateándolos según el porcentaje de participación de cada unidad.

- **RF-12 Integración con plataforma de pago online**

El sistema debe integrar una plataforma de pagos electrónicos (Webpay, Mercado Pago, Khipu, etc.) para permitir a los residentes pagar sus cuotas de gastos comunes directamente desde la plataforma, utilizando tarjetas de crédito o débito.

- **RF-13 Consulta de estado de cuenta del dpto.**

El sistema debe generar y mostrar un registro histórico detallado (cartola) de todos los movimientos financieros de un departamento, incluyendo cargos (emisión de gastos comunes, multas) y abonos (pagos realizados), reflejando el saldo actualizado.

- **RF-14 Identificación y panel de dptos. Morosos**

El sistema debe identificar automáticamente y consolidar en un panel de control a todos los departamentos que mantengan cuotas de ggcc vencidas, permitiendo a la administración visualizar el tiempo de atraso y gestionar el cobro.

- **RF-15 Ingreso de solicitudes y ticket de mantención**

El sistema debe permitir a los residentes y conserjes generar "tickets" formales para reportar problemas de infraestructura en áreas comunes o solicitar reparaciones, centralizando la comunicación y evitando solicitudes informales.

- **RF-16 Actualización y seguimiento de tareas de mantención**

El sistema debe permitir al administrador asignar solicitudes a los encargados de mantención, y permitir a estos últimos actualizar el estado de avance de la tarea, dejando un registro de la solución.

- **RF-17 Registro de incidentes y novedades del edificio**

El sistema debe permitir a los conserjes registrar sucesos relevantes ocurridos durante su turno (robos, ruidos molestos, fallas de seguridad o emergencias), creando una bitácora digital de libre consulta para la administración.

- **RF-18 Registro de visitas y recepción de encomiendas**

El sistema debe permitir a los conserjes registrar el ingreso de visitas y la recepción de paquetes o correspondencia, notificando automáticamente al residente correspondiente para su retiro o autorización.

- **RF-19 Publicación de avisos y circulares informativas**

El sistema debe permitir al administrador redactar y difundir comunicados oficiales, reglamentos o avisos de utilidad pública (cortes de agua, mantención de ascensores, etc.) de forma masiva a todos los residentes.

- **RF-20 Reserva y gestión de espacios comunes**

El sistema debe permitir a los residentes visualizar la disponibilidad y solicitar la reserva de áreas comunes (Quincho, sala de Eventos, etc), gestionando los horarios y el cargo automático del costo de arriendo.

- **RF-21 Historial y bitácora de actividades**

El sistema debe registrar automáticamente cada acción crítica realizada por los usuarios (creación, modificación o eliminación de datos), guardando un historial inmodificable para auditorías posteriores.

- **RF-22 Gestión de respaldo de información**

El sistema debe realizar copias de seguridad automáticas de toda la base de datos y archivos adjuntos (fotos/PDFs) para garantizar la recuperación de la información ante fallas técnicas.

- **RF-23 Búsqueda avanzada y filtros**

El sistema debe proporcionar una herramienta de búsqueda que permita localizar información cruzada (residentes, pagos, unidades, tickets) mediante el uso de múltiples filtros simultáneos.

2.4. Requerimientos no funcionales

- **RNF-01 Seguridad y protección de datos**

El sistema debe garantizar que la información sensible de los residentes esté protegida contra accesos no autorizados.

- **RNF-02 Trazabilidad de acciones**

Capacidad de reconstruir eventos pasados dentro del sistema.

- **RNF-03 Disponibilidad del sistema**

El sistema debe garantizar que el acceso a la plataforma esté operativo la mayor parte del tiempo, asegurando que la gestión de conserjería no se vea interrumpida por fallas técnicas.

- **RNF-04 Recuperabilidad ante fallos**

Capacidad del sistema para restaurar datos en caso de una falla catastrófica del servidor.

- **RNF-05 Usabilidad y diseño responsivo**

La interfaz debe ser intuitiva y adaptarse a computadores, tablets y smartphones.

- **RNF-06 Localización**

El sistema debe estar adaptado al contexto local de Chile.

- **RNF-07 Accesibilidad web**

El sistema debe ser usable por personas con capacidades visuales reducidas.

- **RNF-08 Tiempo de respuesta y rendimiento**

El sistema debe procesar las solicitudes de manera ágil para asegurar la eficiencia en la atención y cobros.

- **RNF-09 Capacidad de almacenamiento**

El sistema debe proveer espacio suficiente para los archivos adjuntos.

- **RNF-10 Escalabilidad de carga**

El sistema debe ser capaz de manejar un aumento en el número de departamentos o residentes sin pérdida de rendimiento.

- **RNF-11 Compatibilidad de navegadores**

La plataforma debe ser accesible desde los navegadores web más utilizados en el mercado chileno.

- **RNF-12 Integridad y consistencia de datos**

El sistema debe garantizar que los cálculos financieros sean exactos y que no se pierda información durante transacciones interrumpidas.

- **RNF-13 Facilidad de mantenimiento**

El software debe estar construido de forma que sea fácil corregir errores o agregar mejoras en el futuro.

- **RNF-14 Manejo de errores**

El sistema no debe cerrarse inesperadamente ante entradas de datos erróneas del usuario.

2.5. Supuestos y dependencias

- **Supuesto:**

Conectividad de Residentes: Se asume que el 100% de los copropietarios y arrendatarios poseen acceso a dispositivos móviles con sistemas operativos vigentes (iOS/Android) o computadores con navegadores web actualizados para interactuar con la plataforma.

Disponibilidad de Datos Históricos: Se asume que la administración actual posee registros (aunque sean manuales) de las deudas históricas, datos de contacto de emergencia y tipos de residencia para la carga inicial del sistema.

Capacidad del Personal: Se asume que los 2 conserjes y las 3 personas de mantención recibirán una capacitación básica para el uso del módulo de solicitudes y seguridad, integrándolo en su rutina laboral diaria.

Integridad de la Red: Se asume que el edificio cuenta con una infraestructura de red estable en áreas comunes para permitir el funcionamiento de funciones en tiempo real, como la apertura remota de portones.

- **Dependencia:**

Plataformas de Pago Externas: El correcto funcionamiento del módulo de "Pagos" depende de la integración exitosa y la estabilidad de APIs de terceros (como Transbank, Webpay o Flow) para el procesamiento de transacciones financieras.

Marco Legal y Normativo: La implementación del sistema de cobros y multas depende de que las reglas de cálculo estén alineadas con el Reglamento de Copropiedad del edificio y la legislación vigente sobre gastos comunes.

Sincronización de Base de Datos: La funcionalidad de visualización de deudas en tiempo real depende de la sincronización constante entre el backend del sistema y el motor de base de datos SQL que gestionará los saldos.

Hardware de Seguridad Existente: La función de "Abrir Portón" depende de la compatibilidad técnica entre el software y los sistemas físicos de automatización (PLCs o controladores de acceso) ya instalados en el edificio.

3. ESTILOS Y PATRONES ARQUITECTÓNICOS

3.1. Estilo arquitectónico adoptado

Para el sistema Vecinos Conectados se ha definido un estilo monolítico con una arquitectura organizada en capas. Esto significa que todos los componentes del software se despliegan como una única unidad lógica, pero su código interno está separado en

capas de Presentación, Lógica de Negocio y Acceso a Datos para mantener el orden y la mantenibilidad.

3.2. Justificación del estilo según el contexto del sistema

Se selecciona este estilo debido a que la complejidad del proyecto (gestión de 160 departamentos) no requiere la sobrecarga de una arquitectura distribuida. Un monolito facilita el desarrollo inicial, simplifica las pruebas de integración y permite una comunicación directa entre los módulos de pagos, visitas y administración sin latencia de red.

3.3. Patrones de diseño aplicados

El patrón de diseño aplicado es MVC.

4. MODELO 4 +1 Y VISTAS ARQUITECTÓNICAS

4.1. VISTA DE ESCENARIO

4.1.1. Propósito

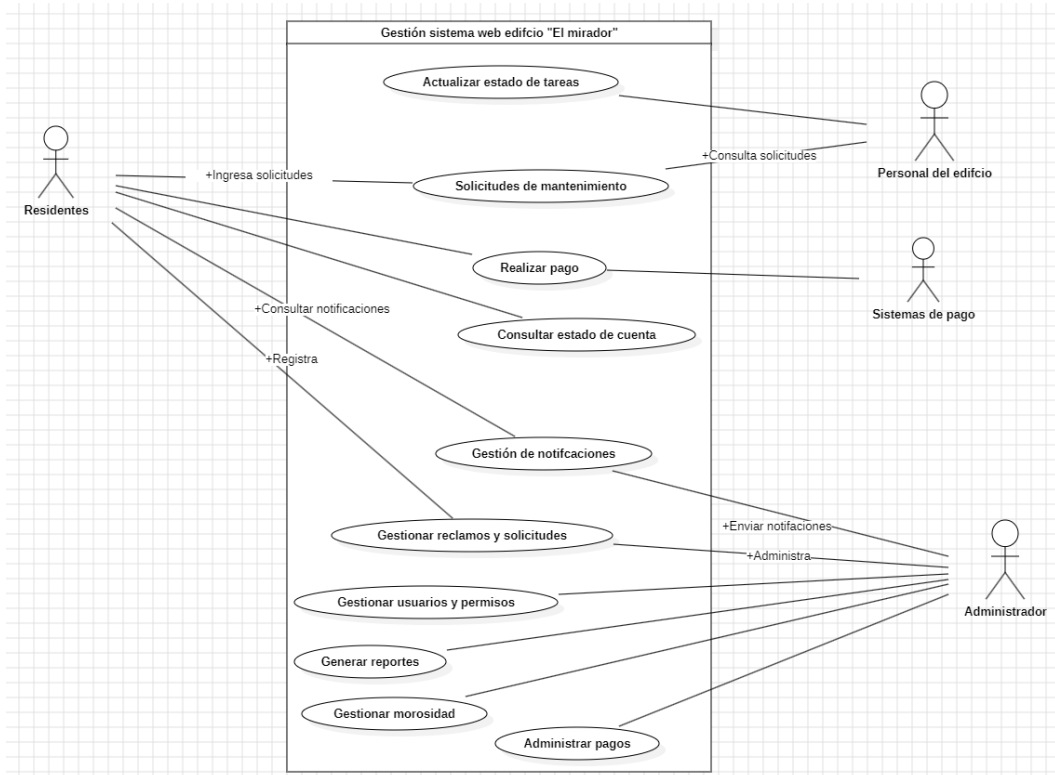
El propósito de esta vista es describir las funcionalidades del sistema desde la perspectiva de los usuarios (Residentes, Administradores y Personal del edificio). Su objetivo es validar que la arquitectura responda a los requerimientos del negocio, como la gestión de pagos y la seguridad del edificio, sirviendo de base para el diseño de las demás vistas técnicas.

4.1.2. Actores

- **Administrador:** Super usuario con permisos totales para gestionar residentes, calcular gastos comunes, asignar roles y generar informes de morosidad
- **Residentes:** Propietarios y arrendatarios que utilizan la plataforma para pagar cuotas, enviar solicitudes de mantenimiento y autorizar visitas.
- **Personal del edificio:** Incluye a conserjes y personal de mantención, quienes interactúan con el sistema para registrar ingresos de visitas y gestionar las solicitudes de reparación de las instalaciones.
- **Sistema de pagos externos:** Entidad externa que procesa las transacciones financieras y confirma el pago de las cuotas al sistema.

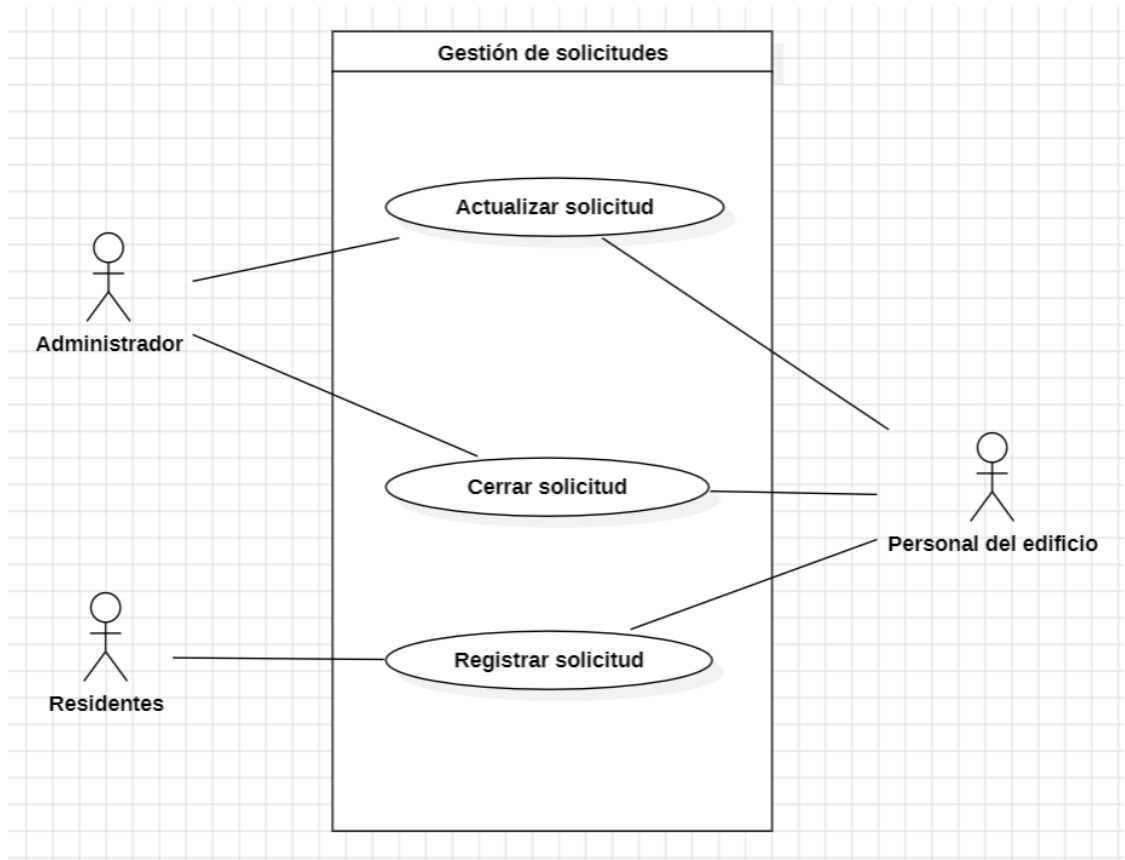
4.1.3. Diagrama general de casos de uso

El diagrama de caso de uso general define la forma en que se comporta el software a través de las interacciones entre los actores y los servicios del sistema.



4.1.4. Diagrama de casos de uso específicos

El caso de uso específico es la gestión de solicitudes de mantenimiento donde los residentes pueden ingresar solicitudes y el personal del edificio recibe las solicitudes, también el personal puede actualizar las solicitudes. El caso de uso cerrar solicitud es una desactivación lógica del registro en la base de datos. El administrador también puede actualizar y desactivar solicitudes.



4.1.5. Lista de casos de uso

Código	Nombre	Actores
CU-01	Actualizar estado de tareas	Personal
CU-02	Solicitud de mantenimiento	Residentes, Personal
CU-03	Realizar pago	Residentes, Sistema de pago
CU-04	Consultar estado de cuenta	Residentes
CU-05	Gestión de notificaciones	Residentes, Administrador
CU-06	Gestión de reclamos y solicitudes	Residentes, Administrador
CU-07	Gestionar usuarios y permisos	Administrador
CU-08	Generar reportes	Administrador
CU-09	Gestionar morosidad	Administrador
CU-10	Administrar pagos	Administrador
CU-11	Registro de solicitudes y mantenimiento	Administrador, Personal del edificio, Residentes
CU-12	Gestión de solicitudes de quejas	Administrador, personal del edificio y residentes
CU-13	Actualizar solicitud	Personal del edificio, Administrador
CU-14	Cerrar solicitud	Personal del edificio, Administrador
CU-15	Registrar solicitud	Residentes

4.1.6. Especificación de casos de uso

Caso de Uso	Registro de solicitudes y mantenimiento	Identificador: CU-11
Actores	Residente (Iniciador), Personal del edificio (Ejecutor) y Administrador	
Tipo	Primario	
Referencias	RF-15 registro de solicitudes de mantenimiento	
Precondición	El usuario debe estar autenticado en la plataforma y pertenecer a una unidad activa	
Postcondición	Se genera un ticket con numero de seguimiento y se notifica al personal responsable	

Descripción	Permite a los residentes fallas en infraestructura o solicitar reparaciones técnicas
Resumen	El residente crea una solicitud, el administrador la asigna y el personal del edificio actualiza el avance hasta el cierre.

CURSO NORMAL

Nro.	Ejecutor	Paso o Actividad
1	Residente	Ingresa al módulo "Solicitudes" y selecciona "crear nuevo ticket"
2	Residente	Describe el problema (ej: filtración) y adjunta evidencia fotográfica si aplica
3	Sistema	Registra solicitud y asigna un estado inicial de "Pendiente"
4	Administrador	Revisa el ticket y lo asigna al personal del edificio
5	Personal del edificio	Actualiza el estado a "En proceso" y finalmente a "Resuelto" tras la reparación.

CURSO ALTERNATIVO

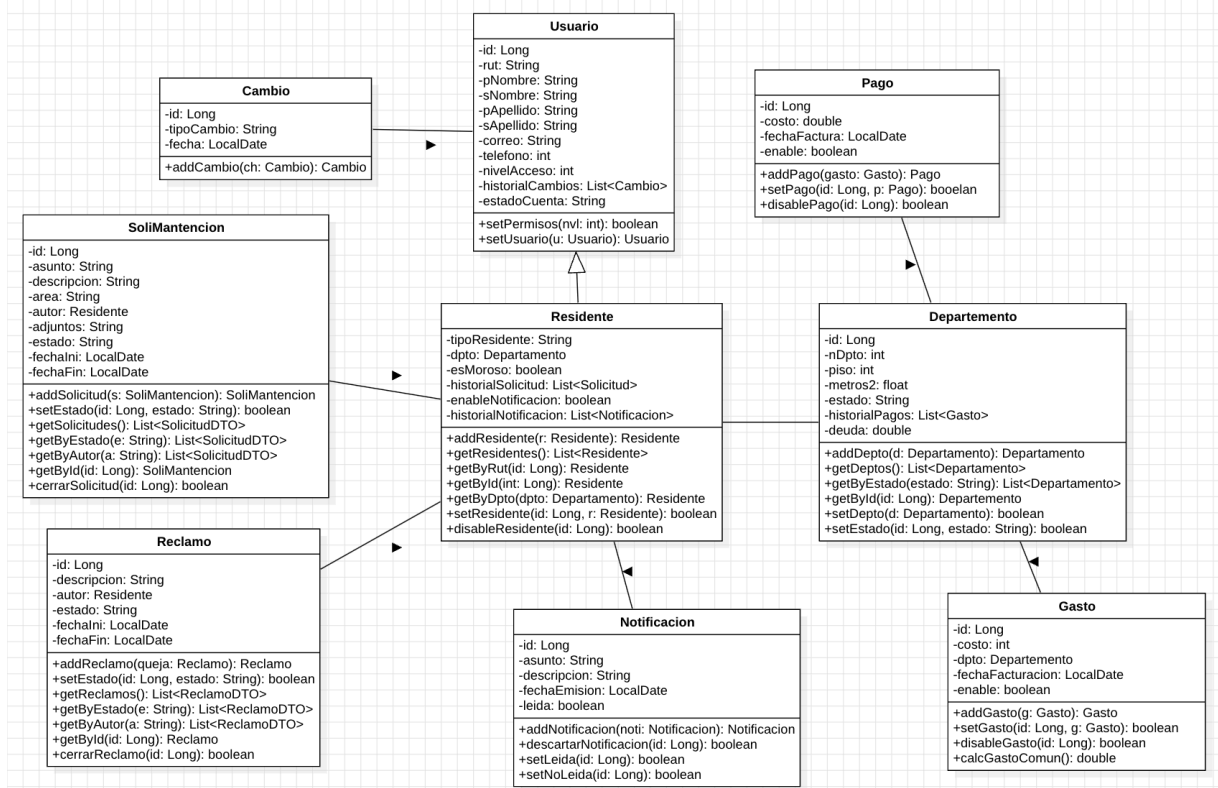
Nro.	Descripción de acciones alternas
1a	Solicitud duplicada: Si ya existe un reporte similar en la misma área común, el sistema sugiere unirse al ticket existente
2a	Rechazo de solicitud: Si la reparación no corresponde a la administración, el administrador cierra el ticket indicando el motivo.
3a	Falta de insumos: El personal marca el ticket como "Pausado" especificando la falta de repuestos o materiales.

4.2. VISTA LÓGICA

4.2.1. Propósito

El propósito de esta vista es modelar la estructura lógica interna del sistema mediante clases y relaciones, permitiendo representar las entidades principales del dominio y su interacción dentro de la solución propuesta.

4.2.2. Diagrama de clases



4.2.3. Descripción diagrama de clases

El diagrama de clases representa la estructura lógica del sistema Vecinos Conectados, modelando las entidades principales, sus atributos y relaciones para soportar la gestión operativa del edificio.

La clase **Usuario** actúa como entidad base del sistema, concentrando atributos comunes como identificación, credenciales y datos de contacto. A partir de esta clase se especializan distintos perfiles mediante herencia, tales como **Administrador**, **Residente** y **Personal**, permitiendo diferenciar permisos y responsabilidades dentro de la plataforma.

La clase **Departamento** representa cada una de las unidades habitacionales del edificio y mantiene asociación con residentes, contratos y registros financieros. Esta relación permite identificar responsables de pago, historial de ocupación y características de cada unidad.

La clase **Pago** modela las transacciones de gastos comunes realizadas por los residentes, almacenando información como monto, fecha, estado y método de pago.

Se relaciona directamente con el residente y el departamento correspondiente, permitiendo trazabilidad financiera y actualización de saldos.

La clase **Solicitud Mantenimiento** representa los tickets generados para reportar incidencias o requerimientos técnicos. Incluye atributos como descripción, prioridad, estado y fecha de creación, además de relaciones con residentes, administrador y personal encargado de resolver la incidencia.

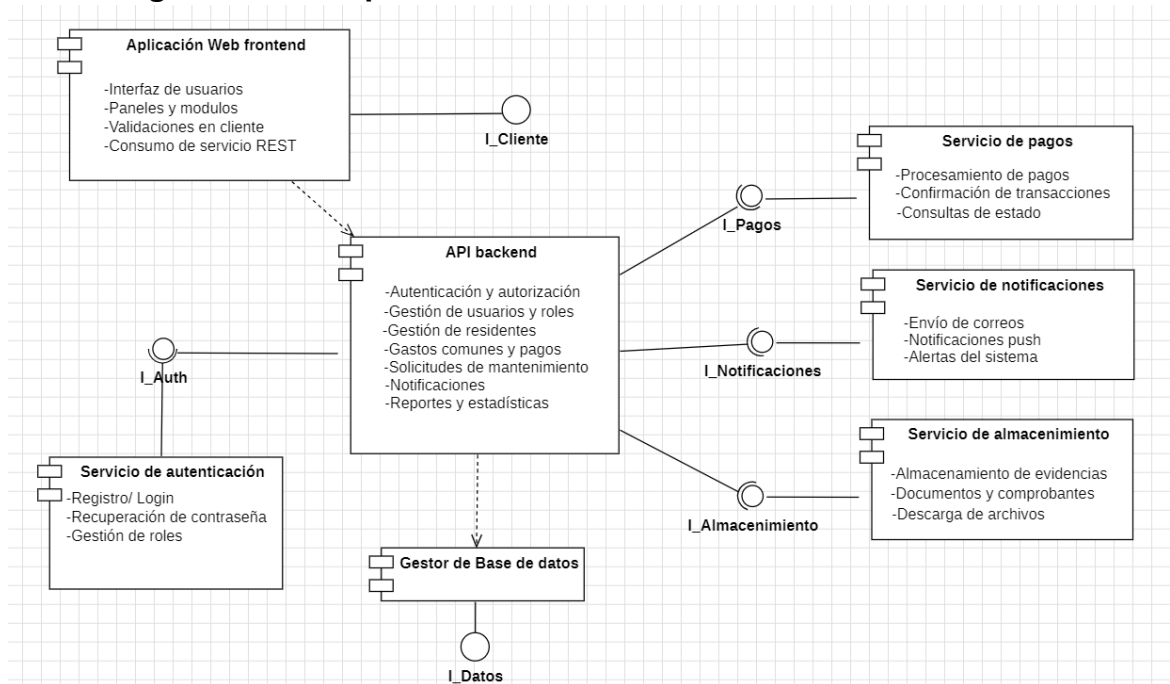
Finalmente, el modelo incorpora relaciones de asociación, herencia y dependencia que permiten estructurar el sistema de forma modular, facilitando la gestión de pagos, mantenimiento, usuarios y control operativo del edificio.

4.3. VISTA DE IMPLEMENTACIÓN/DESARROLLO

4.3.1. Propósito

El propósito de esta vista es describir la organización física de los componentes de software que integran el sistema "Vecinos Conectados". Su objetivo es detallar cómo se estructura el código fuente bajo un esquema monolítico, definiendo la jerarquía de paquetes basada en el patrón MVC y la gestión de dependencias externas para asegurar un desarrollo ordenado y un mantenimiento eficiente de las funcionalidades del edificio.

4.3.2. Diagrama de componente



4.3.3. Descripción diagrama de componente

El diagrama detalla la organización física y lógica de los componentes de software que conforman el sistema Vecinos Conectados, así como sus interfaces de comunicación e integración.

Aplicación Web (Frontend): componente encargado de la interfaz gráfica utilizada por residentes, administración y personal del edificio. Gestiona la interacción del usuario mediante vistas dinámicas y consume los servicios del servidor a través de peticiones REST.

API Backend (Monolito): constituye el núcleo principal del sistema y centraliza la lógica de negocio. Procesa la autenticación de usuarios, la gestión de residentes y departamentos, el cálculo de gastos comunes, el procesamiento de pagos y la generación de reportes.

Servicios internos especializados: módulos organizados dentro del backend que resuelven funcionalidades específicas del sistema:

Servicio de autenticación: controla el acceso seguro a la plataforma, validación de credenciales y asignación de roles y permisos.

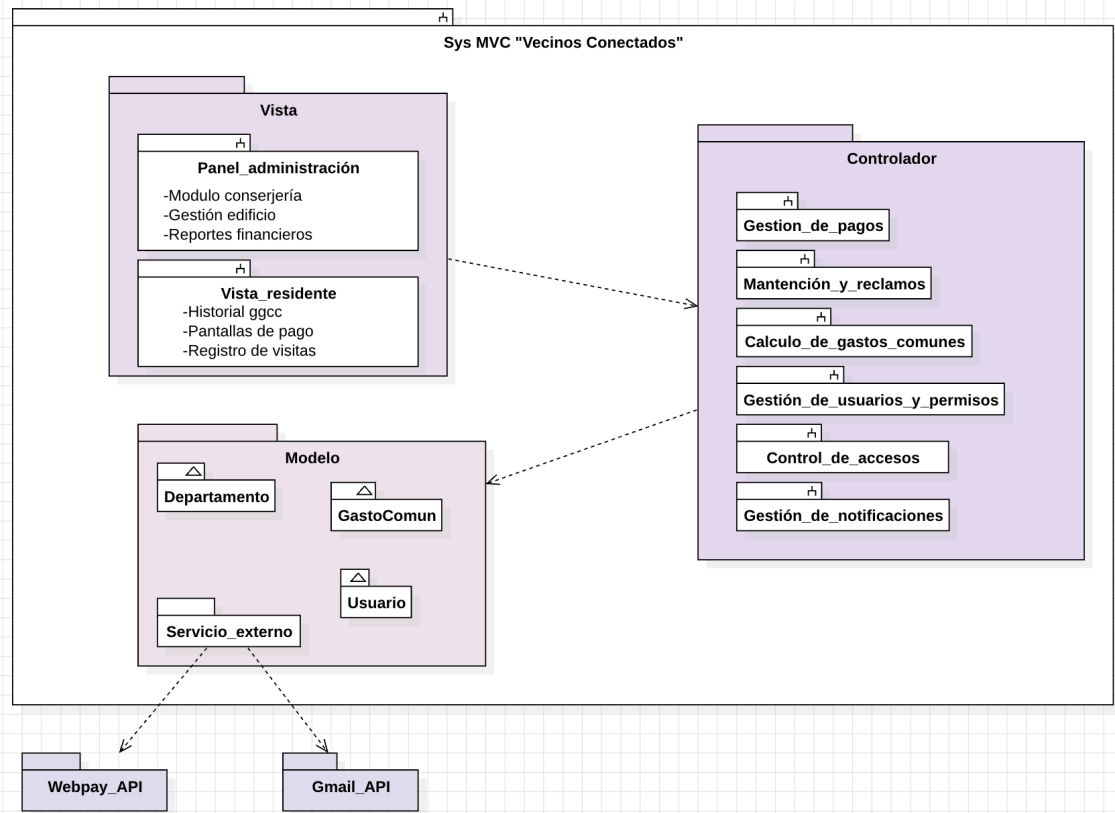
Servicio de pagos: administra el flujo de transacciones financieras e integración con plataformas externas como Webpay o Transbank.

Servicio de notificaciones: gestiona el envío automático de alertas, recordatorios, avisos del sistema y correos electrónicos a residentes.

Servicio de almacenamiento: permite la gestión y persistencia de archivos físicos, tales como evidencias fotográficas, documentos y comprobantes de pago.

Gestor de Base de Datos: componente relacional responsable del almacenamiento persistente e íntegro de la información del sistema, incluyendo usuarios, pagos, solicitudes y registros históricos.

4.3.4. Diagrama de paquete



4.3.5. Descripción diagrama de paquete

El diagrama de paquetes describe la organización interna del monolito "Vecinos Conectados", estructurado bajo el patrón MVC para asegurar una gestión modular de los requerimientos del edificio

Vista: Contiene las interfaces de usuario que permiten la interacción de residentes, administración y personal.

Controlador: Actúa como el núcleo lógico que procesa las solicitudes de la vista y coordina las respuestas mediante la lógica de negocio.

Modelo: Gestiona la persistencia de datos y la integridad de la información del edificio.

Servicio externo: Submódulo del modelo que centraliza la comunicación con plataformas fuera del sistema.

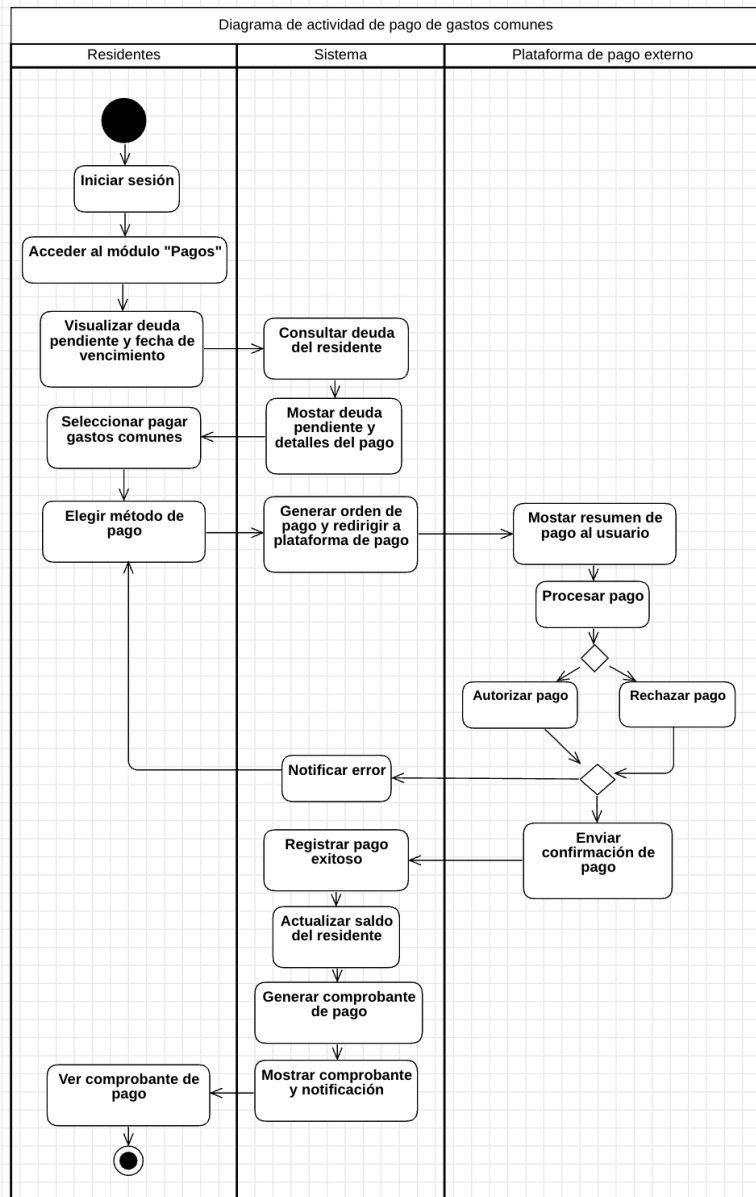
Webpay API (Externo): Representa la pasarela de pagos. La dependencia indica que el monolito delega el procesamiento financiero a este servicio para garantizar transacciones seguras.

4.4. VISTA DE PROCESOS

4.4.1. Propósito

El propósito de este diagrama es modelar el flujo de control y el comportamiento dinámico del sistema ante procesos críticos del edificio. Busca visualizar la secuencia de actividades y decisiones lógicas que ocurren, por ejemplo, durante el flujo de un pago de gastos comunes o la validación de una visita, permitiendo identificar cómo interactúan los módulos del monolito para completar una tarea de forma eficiente.

4.4.2. Diagrama de actividad



4.4.3. Descripción diagrama de actividad

El diagrama modela el comportamiento dinámico y el flujo de control del proceso de recaudación de gastos comunes correspondiente al caso de uso CU-03: Realizar pago. La lógica del proceso se distribuye en cuatro líneas de responsabilidad que interactúan de la siguiente manera:

Residente: inicia el flujo autenticándose en la plataforma y accediendo al módulo de pagos. Luego visualiza su deuda pendiente, selecciona la opción de pago y elige el método correspondiente. Una vez completada exitosamente la transacción, puede visualizar y descargar su comprobante digital.

Sistema: actúa como núcleo central de control del proceso. Se encarga de consultar y mostrar la deuda vigente del residente, generar la orden de pago y coordinar la comunicación entre la plataforma y los servicios externos. Tras recibir la confirmación de pago, registra la transacción, actualiza el saldo del residente y genera el comprobante.

Plataforma de pago externa: recibe la redirección desde el sistema, presenta al usuario el resumen de la operación y procesa la transacción financiera. Posteriormente, envía la respuesta del pago al sistema para continuar con el flujo.

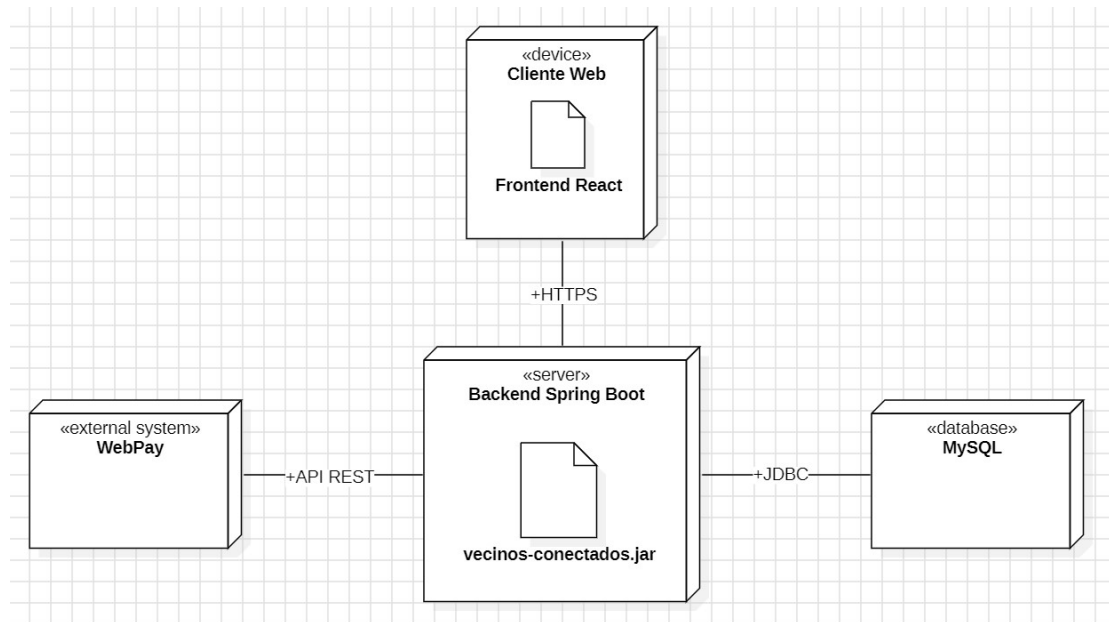
Procesador de pagos: corresponde al servicio bancario encargado de validar y autorizar la operación financiera. Existen dos posibles resultados: autorización del pago, permitiendo finalizar exitosamente la transacción, o rechazo del pago, lo que genera una notificación de error y permite al residente volver a seleccionar un método de pago.

4.5. VISTA FÍSICA

4.5.1. Propósito

El propósito de este diagrama es representar la arquitectura física y la distribución de los componentes del software en el hardware. Detalla la topología de red, los servidores donde se aloja el monolito, el motor de base de datos SQL y la conectividad necesaria para que los 160 departamentos y la administración accedan a la plataforma desde sus dispositivos móviles o computadores.

4.5.2. Diagrama de despliegue



4.5.3. Descripción diagrama de despliegue

El diagrama representa la arquitectura física y la distribución de los componentes de software en la infraestructura de hardware, adoptando un esquema centralizado apto para una arquitectura monolítica:

Dispositivos Cliente (Nodos de Acceso): Representan los computadores y smartphones utilizados por los residentes, la administración y el personal del edificio. El acceso se realiza de forma remota a través de navegadores web mediante el protocolo seguro HTTPS.

Servidor de Aplicaciones (Servidor Web): Nodo central que aloja el monolito ejecutable del sistema. Este servidor procesa todas las solicitudes provenientes de las interfaces (Capa de Presentación) y ejecuta la lógica de negocio (Capa de Control) de manera unificada.

Servidor de Base de Datos: Nodo independiente que contiene el motor de base de datos SQL. Se comunica directamente con el servidor de aplicaciones dentro de la red interna para garantizar una baja latencia en el almacenamiento y consulta de la información de los 160 departamentos.

Servicio Externo (Webpay API): Representa la infraestructura externa de la pasarela de pagos. La conexión se realiza a través de Internet desde el servidor de aplicaciones para procesar de manera segura las transacciones financieras.

5. REQUISITOS DE CALIDAD

5.1. Propósito

5.2. Atributos de calidad (por ejemplo: Usabilidad, Accesibilidad (WCAG), Rendimiento, Mantenibilidad, Seguridad Portabilidad)

ATRIBUTO DE CALIDAD	DESCRIPCION	JUSTIFICACIÓN

5.3. Reglas y criterios de evaluación de calidad

Cómo se medirá el cumplimiento de cada atributo (ej. tiempo de carga < 2 seg, puntuación Nielsen de usabilidad, cumplimiento WCAG nivel AA, etc.).

Herramientas o métodos que se utilizarán(pruebas de carga, pruebas heurísticas, inspecciones, validación con usuarios, etc)

6. PRINCIPIOS DE DISEÑO APLICADOS

6.1. Propósito

6.2. Principios de diseño (por ejemplo: abstracción, acoplamiento, cohesión, encapsulamiento, modularidad)

PRINCIPIO	DESCRIPCIÓN	APLICACIÓN EN EL SISTEMA
Cohesión	Cada módulo o clase tiene una única responsabilidad bien definida.	Los servicios están diseñados para realizar tareas específicas y no múltiples funciones

7. PROTOTIPO

7.1. Propósito

7.2. Mockups (imágenes con una breve descripción)

7.3. Justificar herramientas de prototipado

8. EVALUACIÓN DE CALIDAD HEURÍSTICA DE NIELSEN

8.1. Propósito

8.2. Lista de verificación

8.3. Análisis y métricas de resultados

9. CONTROL DE VERSIONES

9.1. Propósito

9.2. Control de versión utilizado (justificar el tipo de control de versión utilizado (fecha, semántica o secuencial)

9.3. Justificar herramientas de versionamiento

7. CONCLUSIONES